

智造远望 - 智慧工厂可视化大屏

智造远望 - 智慧工厂可视化大屏

项目描述

“智造远望”是一款面向大型制造企业的生产效能与车间调度可视化中台，核心业务涵盖宏观产能大盘监控、车间 AGV 实时调度沙盘以及十万级出入库交叉透视台账。

技术栈

Vue3 + TypeScript + Vite + Turborepo + Pinia + vue-query + ECharts + ZRender + AntV S2 + Ant Design Vue + Less

技术选型

1. 工程基建架构 (Infrastructure & Build)

- **包管理与 Monorepo: PNPM + TurboRepo** 使用 pnpm workspace 进行多包管理，将大屏主应用、后台配置端、通用图表资产与自研监控 SDK 进行物理隔离。利用 TurboRepo 构建智能管道，基于其缓存机制实现跨包的增量构建与多任务并行执行，大幅提升打包效率。
- **构建工具: Vite 5.x** 全面拥抱原生 ESM 生态，为包含海量图表组件与重型渲染库的可视化中台提供极速的冷启动与毫秒级的 HMR（热更新）支持。
- **开发语言: TypeScript 5.x** 实施全链路严格的静态类型检查。利用 Monorepo 的 `packages/shared` 共享全局 TS 类型字典，确保后端海量 WebSocket 推送数据与前端图表 Option 的类型流转绝对安全。

2. 应用核心架构 (Core Application Architecture)

- **前端框架: Vue 3.5 (Composition API)** 采用 `<script setup>` 语法与 Hooks 模式，将图表自适应缩放 (`useResizeObserver`)、全屏状态切换、WebSocket 重连等复杂业务逻辑进行解耦与高度复用。
- **服务端状态调度: 全面引入 @tanstack/vue-query** 接管 HTTP API 轮询。利用其 SWR (Stale-While-Revalidate) 机制与内置的 `refetchInterval`，实现大盘宏观数据（如产能、良品率）的自动缓存、请求去重与后台静默刷新，彻底将服务端状态与客户端状态 (Pinia) 解耦。
- **高频通信逃生舱:** 针对“AGV 小车实时坐标”等极高频场景，搭建 WebSocket 长连接，并在内存层建立独立的数据缓冲池 (Data Buffer)，彻底阻断高频推送数据进入 Vue 响应式 (Proxy) 系统，规避性能污染。

- **路由管理: Vue-Router 4.x** 管理 SPA 单页应用的路由分发，彻底抹平大屏看板区与后台管理区的页面栈跳转，实现全局路由前置守卫与私有化鉴权。

3. 核心视数引擎 (Core Visual & Data Engine)

- **多维基础可视化: ECharts** 承载宏观统计看板与海量传感器时序折线图，深度开启其底层的 `LTTB` (最大三角桶) 降采样算法，在视觉无损的前提下极大地降低主线程内存占用。
- **极限渲染突围 (逃生舱): ZRender** 针对车间海量 AGV 小车高频动态轨迹渲染，避开组件层封装，直接越权调用 ECharts 底层的 ZRender 实例。依靠原生 JS 与纯 Canvas 渲染循环，死守 60fps 极限动画帧率。
- **十万级交叉透视: AntV S2** 摒弃传统的基于 DOM 的 Table 方案，引入蚂蚁金服开源的 Canvas 虚拟表格引擎。彻底解决十万级出入库明细与复杂财务对账单的内存溢出顽疾，并提供类似 Excel 的行列动态交叉透视能力。

4. UI 与视觉体系 (UI & Visual Experience)

- **大屏与中后台 UI: Ant Design Vue** 深度集成阿里系成熟的企业级桌面端组件库，高效构建复杂的后台表单、大盘动态配置面板与弹出层交互，保障 B 端业务流转的严谨性。
- **样式架构与主题: Less** 引入 Less 预处理器，配合原生 CSS Variables，通过深度覆盖底层 `modifyVars` 设计变量，构建了全局暗黑科技主题与大屏绝对定位缩放布局体系。

5. 工程化与质量保障 (Engineering, Quality & DevOps)

- **Lint 规范体系:** `ESLint` + `Prettier` + `EditorConfig`: 统一全团队的代码风格规范与质量底线。
- **Git 工作流规范:** `Husky` + `Lint-staged`: 建立 Git Hooks 卡点，确保只提交符合规范的代码。`Commitlint` + `Commitizen` + `cz-git`: 强制执行 Conventional Commits 提交规范。
- **端侧观测与全链路 APM 基建 (Observability & APM)**
 - **多端轻量级 SDK (@packages/monitor)**: 采用插件化架构设计，在保证对主业务零侵入、低性能影响的前提下，劫持 `window.onerror` 与 `unhandledrejection` 捕获 JS 异常与 Promise 崩溃，并深度提取错误调用堆栈 (Stack Trace) 与用户上下文。同时结合原生 `Performance API` 静默采集页面加载时间、FCP 首屏渲染与核心 API 响应时间。
 - **高并发日志消费链路 (Nest.js + Kafka + ClickHouse)**: 针对大屏 `requestAnimationFrame` 渲染循环可能引发的“瞬时海量报错风暴”，摒弃了传统的直写数据库方案。构建基于 **Nest.js** 的高可用日志接收网关，将上报流量压入 **Kafka** 消息队列进行削峰填谷，最终清洗并落地至 **ClickHouse** 列式数据库。借助 ClickHouse 极高的写入吞吐量与 OLAP 聚合查询能力，为团队提供了分钟级的数据流大盘分析与企微告警闭环。

系统架构设计

1. 宏观架构策略 (Architecture Strategy)

本项目基于 **Monorepo (pnpm workspace + Turborepo)** 架构构建。核心工程理念严格遵循 **"Render-Control Separation" (极速渲染与业务管控解耦)**，确保前台大屏的高频渲染与后台管理的重型交互物理隔离，同时实现底层基建与视数资产的最大化跨端复用。

- **包管理器:** `pnpm` (利用硬链接与 Workspace 机制解决依赖幽灵与版本碎片化)
- **构建管道:** `Turborepo` (基于缓存机制实现跨包增量构建与多任务并行)
- **技术底座:** `Vue 3.5+` (Composition API) + `Vite 5.x` + `TypeScript 5.x`
- **样式架构:** `Less` + 原生 CSS Variables (构建统一的 Design Token 与全局暗黑模式)

2. 工程物理目录拓扑 (Directory Topology)

系统划分为 `apps` (业务应用层) 与 `packages` (共享基础设施层) 两大核心工作区：

```
代码块

1  smart-manufacturing-mono/
2  |— package.json           # 根级配置 (husky, commitlint, lint-staged)
3  |— pnpm-workspace.yaml    # 工作区声明 (apps/*, packages/*)
4  |— turbo.json             # Turbo 管道任务编排 (build, lint, dev)
5  |— tsconfig.base.json     # 全局 TypeScript 基准配置
6  |
7  |— apps/                  # ===== 业务应用层
   =====
8  |   |— dashboard/        # 【大屏视数中心】(生产安全与态势感知大盘)
9  |   |   |— package.json  # 依赖: @packages/charts, shared, monitor,
   config
10 |   |   |— vite.config.ts # 独立分包策略 (ECharts/ZRender Chunk 抽离)
11 |   |   |— src/
12 |   |       |— main.ts    # 入口: 注册纯净监控探针
13 |   |       |— views/    # 宏观产能页、AGV 实时沙盘页
14 |   |       |— router/   # 单页路由分发
15 |   |
16 |   |— admin/            # 【中后台管控端】(台账透视与系统运维后台)
17 |       |— package.json  # 依赖: @packages/charts(部分), shared,
   monitor
18 |       |— vite.config.ts # Ant Design Vue 组件按需加载配置
19 |       |— src/
20 |           |— layout/   # B 端标准侧边栏视图容器
21 |           |— views/    # 出入库透视表、设备管控与监控日志台账
22 |           |— router/   # 路由体系与 RBAC 动态权限守卫
23 |
24 |— packages/              # ===== 共享基础设施层 =====
   |— charts/                # 【核心视数物料库】
26 |   |— package.json      # 依赖: echarts, zrender, @antv/s2-vue
27 |   |— src/
28 |       |— echarts/      # 基于 LTTB 降采样算法的时序图表封装
```

```

29      |      |—— zrender/      # 脱离 Vue 响应式的 AGV 纯 Canvas 渲染池
30      |      |—— s2/          # 十万级虚拟滚动交叉透视表 (Pivot Table)
31      |
32      |—— shared/              # 【通信与状态中枢】
33      |   |—— package.json     # 依赖: pinia, @tanstack/vue-query, axios
34      |   |—— src/
35      |       |—— store/        # Pinia 客户端全局状态 (UI 交互与鉴权)
36      |       |—— network/      # Vue Query 宏观低频数据轮询引擎
37      |       |—— websocket/    # 原生 JS Map 内存缓冲池 (Data Buffer)
38      |       |—— types/        # 全局高复用 TypeScript 协议 (IPivot, IAgv)
39      |
40      |—— monitor/             # 【轻量级端侧观测 SDK】(无任何第三方依赖)
41      |   |—— package.json     # 独立纯 TS 模块
42      |   |—— index.ts         # 暴露全局初始化方法 initMonitor()
43      |   |—— src/
44      |       |—— error-catch.ts # 运行时异常捕获 (window.onerror /
unhandledrejection)
45      |       |—— perf-observe.ts # Web Vitals 采集 (FCP, LCP, TTFB)
46      |       |—— reporter.ts    # 异步低损耗上报 (navigator.sendBeacon)
47      |
48      |—— config/              # 【工程化规范与设计体系】
49      |   |—— eslint/          # ESLint 统一代码质量底线
50      |   |—— typescript/      # 跨包共享的基础 TSConfig
51      |   |—— styles/          # Less 全局变量、Mixins 与主题系统

```

3. 核心流转与微架构规范 (Micro-Architecture & Data Flow)

系统在运行时，底层数据与渲染链路被严格划分为三条互不干扰的独立管道：

● 管道一：服务端状态与宏观视图 (HTTP Polling)

- **场景：**低频大盘数据（产能总览、月度良品率）。
- **机制：**通过 `@packages/shared` 中的 `@tanstack/vue-query` 发起 HTTP 请求。利用其 SWR (Stale-While-Revalidate) 缓存机制接管数据流，并配置 `refetchInterval` 实现后台静默刷新，彻底解耦 Pinia。

● 管道二：极限渲染与物理沙盘 (60fps Canvas Loop)

- **场景：**车间海量 AGV 小车高频坐标更新。
- **机制：**建立 WebSocket 长连接通道。高频报文进入前台后，**强制拦截并写入原生 JS Map 内存池**，绝对禁止触发 Vue Proxy 深度劫持。底层开启原生 `requestAnimationFrame` 循环，每 16.6ms 读取一次内存池，直接越权驱动 `@packages/charts/zrender` 中的图形实例进行重绘。

● 管道三：旁路观测与排障闭环 (Telemetry SDK)

- **场景：**大屏极速运行时的性能损耗与静默异常兜底。

- **机制：** `@packages/monitor` 作为极简旁路探针静默运行。当捕获到 Canvas 崩溃或 FCP 耗时超标时，立即进行本地脱敏与格式化，利用 `navigator.sendBeacon` 将日志压入后端高并发接收队列，实现零业务侵入的端侧观测闭环。

项目职责

- **业务开发：** 依托 Monorepo 体系负责大屏视数中心与后台管控端核心模块落地。打通“大盘监控-沙盘调度-台账透视”完整链路，实现底层逻辑与视数资产库的高效跨端复用，保障业务稳健交付。
- **极限渲染：** 独立攻坚 AGV 调度沙盘 60fps 极限动效。摒弃 Vue 响应式劫持，构建原生 JS 内存缓冲池承接 WebSocket 高频坐标，越权直调 ZRender 实例进行纯物理重绘，彻底根除海量节点卡顿。
- **表格破局：** 负责后台十万级出入库台账与复杂财务对账单开发。引入蚂蚁 AntV S2 虚拟 Canvas 表格引擎替代传统 DOM 方案，彻底解决多维交叉透视与海量数据无尽滚动时的内存溢出白屏死结。
- **状态调度：** 重构全局网络与状态分治引擎。引入 Vue Query 接管大屏宏观数据的 HTTP 静默轮询与 SWR 缓存，与 Pinia 客户端纯交互状态彻底解耦，大幅降低复杂视数场景下的组件无意义重渲染。
- **物料封装：** 沉淀并抽离 15+ 高频工业级视数图元组件。针对单月海量传感器时序折线图，深度开启 ECharts 底层 LTTB 降采样算法，在视觉无损前提下将节点渲染量压缩逾 90%，极大释放主线程压力。
- **端侧观测：** 针对私有化物理隔离内网，独立自研极简无依赖的前端监控 SDK。底层静默劫持全局异常并结合 Performance API 采集首屏 FCP，统一上报内网日志网关，以极低成本闭环大屏静默故障排查。

项目难点与亮点

面试高频问题